

ITSS SOFTWARE DEVELOPMENT/ SOFTWARE DESIGN AND CONSTRUCTION

Lab 00: Release Flow Guidelines

Lecturer: NGUYEN Thi Thu Trang, trangntt@soict.hust.edu.vn

1. BRANCH YOUR REPOSITORY

Day after day, your repository becomes more and more sophisticated, which makes your codes harder to manage. Luckily, a Git workflow can help you tackle this. A Git workflow is a **recipe for how to use Git** to control source code in a consistent and productive manner. Release Flow¹ is a lightweight but effective Git workflow that helps teams cooperate with a large size and regardless of technical expertise.

Applying Release Flow is required for every group lab in this course.

However, we would use a modified version of Release Flow for simplicity.

- We can create as many branches as we need.
- We name branches with meaningful names. See Table 1-Branching policy.
- We had better **keep branches as close to master as possible**; otherwise, we could face merge hell.
- Generally, when we merge a branch with its origin, that branch has been history. We usually do not touch it a second time.
- **We must strictly follow the branching policy. Others are flexible.**

¹ <https://docs.microsoft.com/en-us/azure/devops/repos/git/git-branching-guidance?view=azure-devops>

Branch	Naming convention	Origin	Merge to	Purpose
feature or topic	+ feature/feature-name + feature/feature-area + topic/description	master	master	Add a new feature or a topic
bugfix	bugfix/description	master	master	Fix a bug
		feature	feature	
hotfix	hotfix/description	release	release & master [1]	Fix a bug in a submitted assignment after deadline
refactor	refactor/description	master	master	Refactor the project's structure...
		feature	feature	
release	release/labXX	master	none	Submit assignment [2]

Table 1: Branching policy

[1] If you want to **update your solutions within a week after the deadline**, you could make a new hotfix branch (e.g., hotfix/stop-the-world). Then, merge the hotfix branch with master and with release branch for the last submitted assignment (e.g., release/lab01). In case you already created a release branch for the current week assignment (e.g., release/lab02), we could merge the hotfix branch with the current *release branch if need be*, or we can delete and then recreate the current release branch.

[2] **Latest versions of projects in release branch serve as the submitted assignment.**

Let's use Release Flow as our Git workflow and apply it to refactor our repositories.

Step 1: Create a new branch in our local repository.

Step 2: Make our changes, test them, and push them into the previous branch that you have created.

See <https://www.atlassian.com/git/tutorials/undoing-changes> to undo changes in case of problems.

To improve the commit message, see <https://thoughtbot.com/blog/5-useful-tips-for-a-better-commit-message>.

Step 3: Make a pull request for reviews from our teammates². We **skip** this step since we are **solo** in this repository. We, however, had better never omit this step when we work as a team.

Step 4: Merge branches.

The result is shown in the following figure.



Hints:

Typical steps for a new branch:

- Standing in the master/main branch, create and switch to a new branch (e.g. abc) in the local repo:

```
(master) $ git checkout -b <name of the branch>
```

 For example:

```
(master) $ git checkout -b feature/apply-release-flow
```
- Make modification in the local repo.
- Add all changes in the current directory and its subdirectories to the staging area:

```
(feature/apply-release-flow) $ git add .
```
- Commit the change in the local repo:

```
(feature/apply-release-flow) $ git commit -m "commit message"
```
- Push the local branch to the remote branch:

```
(feature/apply-release-flow) $ git push origin feature/apply-release-flow
```
- Merge the remote branch (e.g. refactor/apply-release-flow) to the master branch (Using “Pull request” function in Github)

After completing all the tasks of that week, and merge all branches into master/main branch, you should create a release/labxx branch from the master in the remote repo (GitHub).

Notice: From this step, I assume that you have completed all the tasks from the previous step, such as refactoring the repository format in the previous step.

² <https://www.atlassian.com/git/tutorials/making-a-pull-request>

2. RELEASE FLOW DEMONSTRATION

2.1 Hypothesis

We hypothesize that Figure 1 shows the branches of our current remote repository.

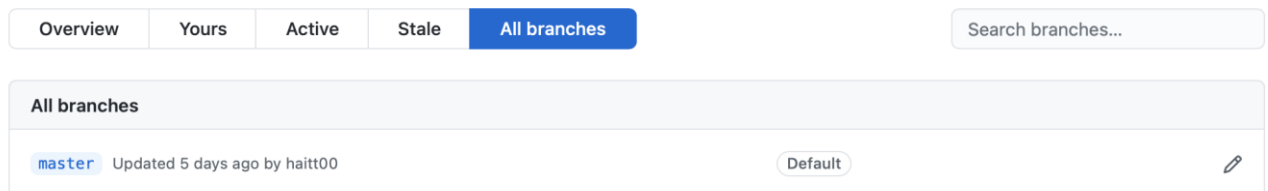


Figure 1. Branches of Remote Repository

Now we add a new topic or a new feature to our application. The next section shows us how to apply Release Flow in this hypothesis.

2.2 Demonstration

Step 1. Update local repository: Issue the following command and resolve conflicts if any.

```
(master) $ git pull origin master
```

Step 2. Create and switch to a new branch in the local repository:

```
(master) $ git checkout -b feature/demonstrate-release-flow
```

Step 3. Make modifications in the local repository. Here, you will make changes to the files in the local repository, and then save those changes (for example: adding new files, modifying file contents, deleting files...).

Step 4. Add all changes in the current directory and its subdirectories to the staging area:

```
(feature/demonstrate-release-flow) $ git add .
```

Or you can add a specific file:

```
(feature/demonstrate-release-flow) $ git add
tkxdpm/documents/ReleaseFlow.txt
```

Step 5. Commit the change in the local repository:

```
(feature/demonstrate-release-flow) $ git commit -m "done feature
demonstrate release flow"
```

Step 6. Push the local branch to the remote branch:

```
(feature/demonstrate-release-flow) $ git push origin
feature/demonstrate-release-flow
```

Step 7. Create a pull request in GitHub GUI (for working in a team only):

Firstly, choose the “Pull requests” tab from the top navigation bar.

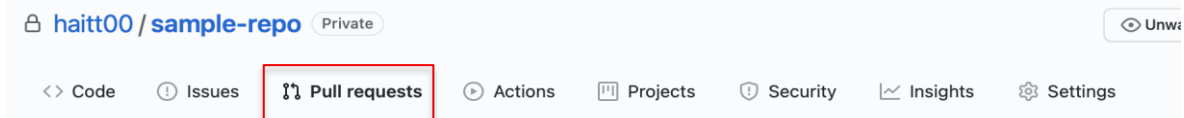


Figure 2. Creation of a Pull Request in GitHub GUI (1/4)

Secondly, click the button “New pull request” in the top right corner of the interface.

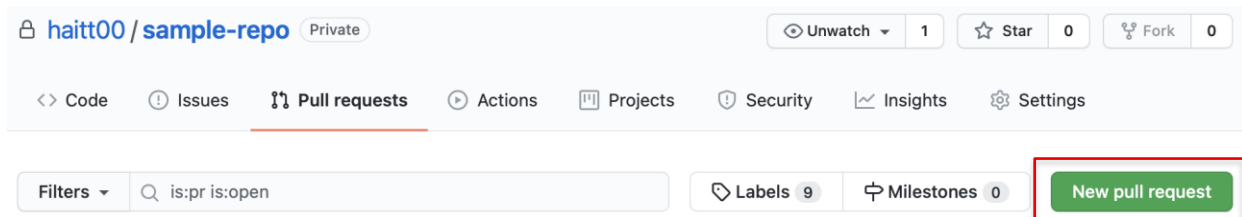


Figure 3. Creation of a Pull Request in GitHub GUI (2/4)

Then, pick the target branch and current branch. Besides, at the bottom of the interface, we can see the changes between the current branch and the target branch. Choose “Create pull request” to the top right. Notice that: The target branch will affect the destination branch which we want our branch to merge in the next step.

Comparing changes

Choose two branches to see what's changed or to start a new pull request. If you need to, you can also [compare across forks](#).

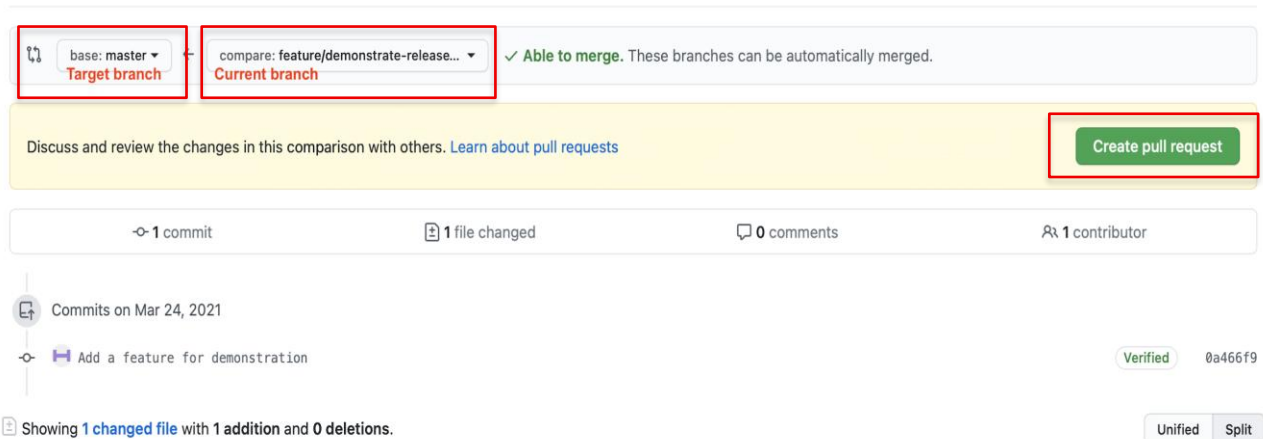


Figure 4. Creation of a Pull Request in GitHub GUI (3/4)

Lastly, choose reviewers for the pull request. We can also change the commit message and add comments as we desire. Choose “Create pull request”

Open a pull request

Create a new pull request by comparing changes across two branches. If you need to, you can also [compare across forks](#).

The following figure shows the result of our efforts in the dashboard of GitHub. The added reviewers also can see the pull requests in their dashboard. When the changes are viewed, we can merge the branches.

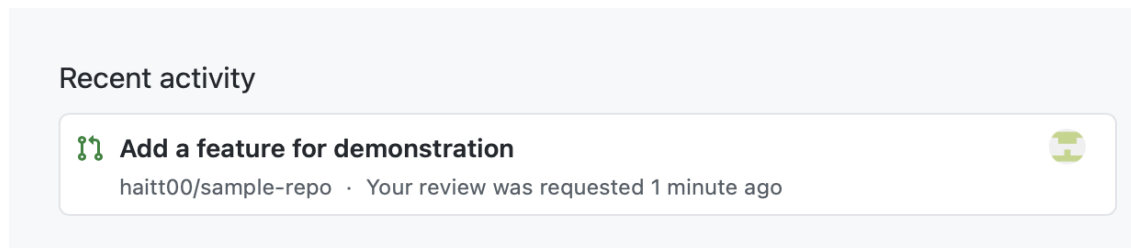


Figure 5. Creation of a Pull Request in GitHub GUI (4/4)

Step 8. Merge the new remote branch to the master branch:

- Open the pull request.
- Choose “Merge pull request”. You can choose one of several merge options from the drop-down menu

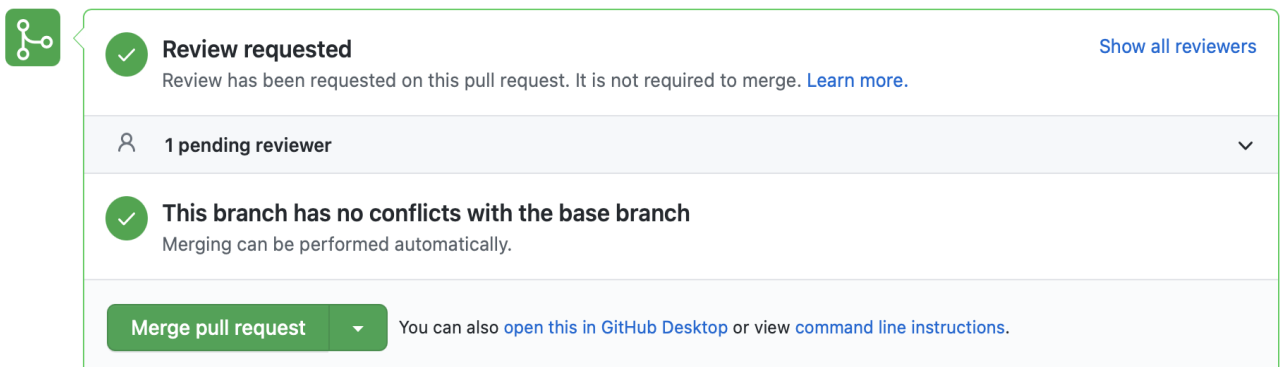


Figure 6. Branch merging (1/3)

Lastly, change the commit message if need be. We cannot change the destination branch. Choose “Confirm merge” (as shown in Figure 6)

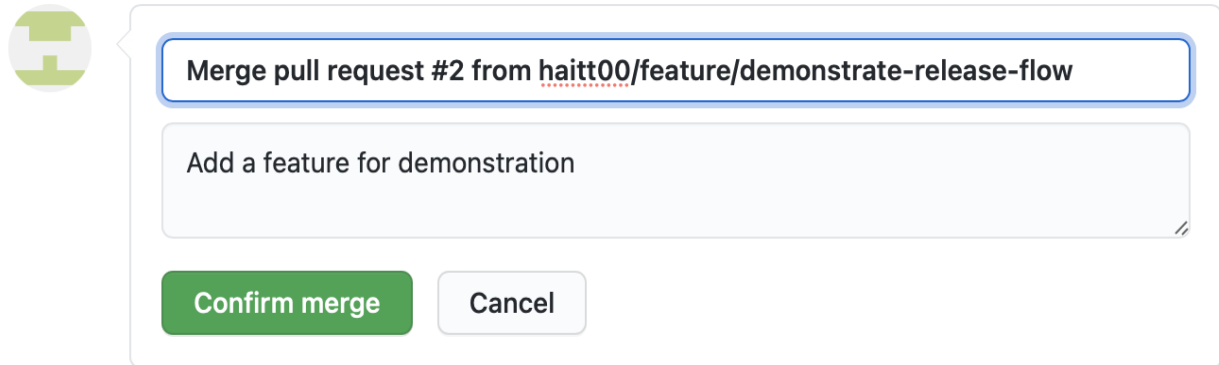


Figure 7. Branch merging (2/3)

The following figure shows the result of our efforts. The changes from the target branch have been merged to the target branch “master”.

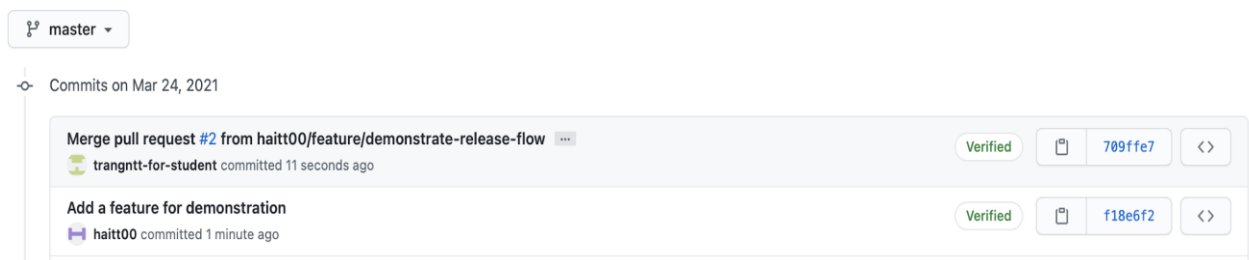


Figure 8. Branch merging (3/3)